

Trajectory-Aware RL for Efficient Multi-Hop Retrieval

Oğuz Kağan Hitit • Eren Yavuz

Koç University

COMP438 Final Project Report

GitHub

Spring 2025

1 Introduction

Large Language Models (LLMs) have become key components of retrieval-augmented generation (RAG) pipelines, where generating effective queries is essential for retrieving relevant evidence to support downstream reasoning tasks. Despite their strong performance, state-of-the-art approaches [1, 2, 3] often rely on instruction-tuned models with billions of parameters. While such large models excel at complex reasoning and retrieval, they come with substantial computational costs, memory requirements, and deployment challenges, making them impractical for resource-constrained settings.

Multi-hop retrieval presents additional challenges beyond single-step retrieval. It requires iteratively refining the model’s understanding of the query, where each step depends heavily on the success of previous retrievals. A single poorly formulated query can propagate errors, leading to irrelevant documents and degraded overall performance. This sequential dependency makes multi-hop retrieval more akin to a planning problem, where the model must not only generate accurate individual queries but also coordinate a trajectory of queries that collectively surface the necessary evidence.

To address these challenges, we ask: *Can small language models, when optimized through reinforcement learning, achieve competitive performance for multi-hop query generation?* We focus on lightweight models such as google/flan-t5-small (77M parameters) and explore preference-based reinforcement learning (RL) techniques, specifically Identity Policy Optimization (IPO), to enhance their retrieval effectiveness. Our experiments are conducted on the HotpotQA dataset [4], a benchmark designed for multi-hop reasoning and evidence retrieval. By targeting smaller models, we aim to reduce computational overhead while maintaining, or even improving retrieval quality within RAG pipelines.

Our main contributions can be summarized as follows:

1. We propose a trajectory-aware multi-hop retrieval framework optimized with preference-based RL (IPO) for small-scale query generators.
2. We design a preference dataset sampling pipeline that leverages multiple language models to create robust pairwise comparisons.
3. We demonstrate that a lightweight query generator trained with IPO achieves up to an 8% improvement in Average Precision (AP) over few-shot baselines.

2 Preliminaries

In this section, we briefly review two key components of our framework: Identity Policy Optimization (IPO), a preference-based reinforcement learning method, and ColBERTv2, a late-interaction dense retriever that we use to evaluate query quality.

2.1 Identity Policy Optimization (IPO)

Preference-based reinforcement learning (RL) is an alternative to standard scalar-reward RL, where instead of using explicit numeric rewards, the agent learns from pairwise comparisons between outputs. Identity Policy Optimization (IPO) [5, 6] is a recently proposed preference-based optimization method that is both stable and data-efficient.

Unlike traditional policy gradient methods such as REINFORCE or PPO, IPO does not require estimating advantage functions or sampling from sparse reward distributions. Instead, it directly optimizes the policy to increase the likelihood of preferred outputs relative to dispreferred ones.

Given an input x (e.g., a question and retrieval context), and two candidate outputs y^+ (preferred) and y^- (dispreferred), IPO minimizes:

$$\mathcal{L}_{\text{IPO}} = \left(\log \pi_{\theta}(y^+ | x) - \log \pi_{\theta}(y^- | x) - \frac{1}{2\tau} \right)^2,$$

where $\tau > 0$ is a temperature parameter that controls the target margin between preferred and dispreferred outputs. This loss encourages the model to learn a relative ranking of outputs consistent with the preference signal, rather than focusing on absolute probabilities.

2.2 ColBERTv2 Retriever

We evaluate the quality of generated queries using ColBERTv2 [7], a late-interaction dense retriever designed for fine-grained token-level matching. ColBERTv2 improves upon traditional dense retrieval methods (e.g., sentence-level embeddings) by retaining token-level information during similarity computation, enabling more precise ranking of documents.

ColBERTv2 encodes both queries and documents into sequences of contextualized token embeddings using a transformer encoder. At retrieval time, it computes the similarity between a query q and a document d as:

$$\text{Sim}(q, d) = \sum_{t \in q} \max_{s \in d} \langle \mathbf{q}_t, \mathbf{d}_s \rangle,$$

where $\mathbf{q}_t$ and $\mathbf{d}_s$ are the embedding vectors of query token t and document token s , and $\langle \cdot, \cdot \rangle$ denotes cosine similarity. This token-level max-sim approach allows ColBERTv2 to capture partial matches and nuanced term importance, making it particularly well-suited for multi-hop retrieval tasks where precise document matching is critical.

By using ColBERTv2 as our retriever, we can compute evaluation metrics such as Average Precision (AP) based on how well the top- k documents retrieved by a query align with the gold supporting documents in HotpotQA.

3 Related Work

3.1 Reinforcement Learning for RAG

Recent approaches have applied reinforcement learning to optimize retrieval-augmented generation (RAG) pipelines, aiming to jointly improve query generation, retrieval, filtering, and answer synthesis. For example, MMOA-RAG [8] employs a collaborative multi-agent RL framework to jointly optimize multiple stages of RAG, query rewriting, retrieval, filtering, and answer generation, using a shared reward signal, achieving notable improvements on multi-hop QA tasks. RAG-RL [2] fine-tunes a reasoning LLM with GRPO, curriculum learning, and rule-based rewards, reaching state-of-the-art results on datasets such as HotpotQA and MuSiQue. In the biomedical domain, RL-based RAG approaches [9] focus on producing layperson-friendly summaries by jointly optimizing both retrieval accuracy and readability through multi-objective training. Similarly, M-RAG [3] introduces a partitioned multi-

agent RL pipeline, aligning retrieval and generation objectives to achieve strong cross-domain generalization.

3.2 Learning to Retrieve by Trying (LeReT)

LeReT [1] also approaches query generation in RAG as a reinforcement learning problem. At each retrieval step, a query generator π_r receives the original user question u along with the accumulated retrieval context C_{h-1} and produces a new query $q_h \sim \pi_r(\cdot | u, C_{h-1})$. A retriever $\mathcal{K}$ then returns a set of documents $D_h = \mathcal{K}(q_h)$, and the quality of this retrieval is measured using a reward $r(u, C_{h-1}, q_h)$ based on Average Precision (AP) relative to gold supporting facts. LeReT trains the query generator through preference-based optimization. Instead of relying on traditional policy gradient techniques or a separate learned reward model, LeReT directly fine-tunes π_r using Identity Policy Optimization (IPO), which aligns the model with the preferred queries in a stable and data-efficient manner.

However, LeReT optimizes large language models, specifically Gemma2-9B and LLaMA3-8B to achieve its results. While they prove the usefulness of their approach, it comes with significant computational costs, motivating our investigation into whether smaller language models can achieve improved performance when optimized with similar RL techniques.

4 Our Approach

4.1 Task Formulation

We formulate multi-hop query generation for RAG as a Markov Decision Process (MDP), where the query generator acts as a policy π_θ with parameters θ . The objective is to learn π_θ such that it produces a sequence of queries that efficiently retrieves the necessary supporting documents.

At a high level, this setting naturally maps to reinforcement learning because multi-hop retrieval involves sequential decision-making: each query (action) influences the set of retrieved documents (state transition), which in turn affects the quality of future queries. Like a planning problem, the model must not only make high-quality individual queries but also plan a trajectory of queries that collectively maximize retrieval performance. If an early query is suboptimal, it can lead to a chain of irrelevant documents, compounding errors and reducing the success of subsequent hops, similar to how an RL agent making poor early moves in a game may jeopardize its final reward.

4.1.1 State Space

At each retrieval step h , the state $s_h \in \mathcal{S}$ represents the cumulative retrieval history:

$$s_h = (q_0, \{q_i\}_{i=1}^{h-1}, \{D_i\}_{i=1}^{h-1}),$$

where:

- q_0 is the user prompt,
- $\{q_i\}_{i=1}^{h-1}$ are all previously generated queries,
- $\{D_i\}_{i=1}^{h-1}$ are the document sets retrieved for each previous query.

4.1.2 Action Space

At each hop h , the action $a_h \in \mathcal{A}$ is generating the next search query q_h . The policy π_θ outputs a probability distribution over possible queries conditioned on the current state:

$$a_h = q_h \sim \pi_\theta(s_h)$$

The generated query q_h is then passed to the retriever, which returns a set of documents D_h .

4.1.3 Transition Function

The environment transitions to the next state by appending the new query and its retrieved documents:

$$s_{h+1} = (q_0, \{q_i\}_{i=1}^h, \{D_i\}_{i=1}^h) = s_h \cup \{q_h, D_h\}$$

4.1.4 Reward Function

To optimize π_θ , we define a reward function that with a discount factor. We compute a cumulative discounted reward:

$$R = \sum_{h=1}^H \gamma_h \cdot \text{AP}(D_h),$$

where:

- $\text{AP}(D_h)$ denotes the Average Precision (AP) of the retrieved set D_h with respect to gold supporting documents,
- $\gamma_h \in (0, 1]$ is the discount factor of hop h .

4.1.5 Policy and Objective

Our goal is to optimize π_θ to maximize the expected cumulative discounted reward over the entire retrieval trajectory:

$$\max_{\theta} \mathbb{E}_{\pi_\theta} \left[\sum_{h=1}^H \gamma_h \cdot \text{AP}(D_h) \right]$$

To do that, we use Identity Policy Optimization (IPO) [5, 6].

4.2 Preference Dataset Creation

A high-quality preference dataset is essential for training our query generator with Identity Policy Optimization (IPO). Instead of using explicit scalar rewards, IPO relies on pairwise comparisons that indicate which query performs better. Therefore, we need a dataset that not only provides diverse candidate queries but also includes reliable signals for ranking these queries.

We build our preference dataset on top of the HotpotQA benchmark [4], a multi-hop question answering dataset designed to test reasoning across multiple supporting documents. Each HotpotQA instance includes:

- A question that requires reasoning over at least two evidence documents,
- Gold supporting facts with document titles and sentence-level annotations identifying the required evidence,
- A context, listing all documents and their sentences for the question.

For our experiments, we sample 5,000 training examples and 1,000 examples for both validation and testing. Because each question involves two retrieval hops, we generate preference pairs for each hop separately, which allows us to evaluate query generation performance at different stages of retrieval.

As illustrated in Figure 1, we build the preference dataset through three key stages: candidate query generation, document retrieval, and preference pair construction. For each HotpotQA question, we generate $k = 5$ candidate queries using a 3-shot prompting strategy. We use the prompt template shown in Figure 2, that includes 3 randomly selected question-query examples sampled from a pool of 100 pairs produced by GPT-4o. For each query, we retrieve the top- k documents ($k = 5$) from the set of documents associated with the given HotpotQA question. ColBERTv2 computes similarity scores by comparing contextualized token embeddings and aggregating the maximum cosine similarities across query tokens, enabling ranking of the retrieved documents. To establish preference labels, we compute the Average Precision (AP) score of each query’s retrieved documents against the gold supporting documents. For every pair of candidate queries (q_i, q_j) , we determine the preferred query based on AP:

$$(q_i, q_j) \rightarrow \begin{cases} q_i \succ q_j & \text{if } \text{AP}(q_i) > \text{AP}(q_j), \\ q_j \succ q_i & \text{otherwise.} \end{cases}$$

This step results in a collection of pairwise comparisons that encode which queries are more effective for multi-hop retrieval.

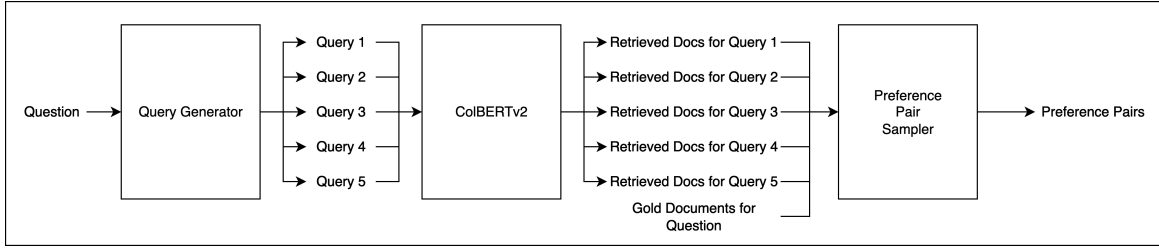


Figure 1: Preference dataset creation pipeline for each sample in HotpotQA dataset

4.3 Experimental Setup

We evaluate one baseline and four trained configurations:

- **BASELINE (FEW-SHOT ONLY):** This setup evaluates the original google/flan-t5-small model using 3-shot prompting, without any preference dataset creation or RL-based training.

We train google/flan-t5-small with preference datasets created by different query generators using IPO:

1. **w/T5:** Using google/flan-t5-small itself as the query generator to create preference pairs.
2. **w/LLAMA3-8B:** Using meta-llama/Llama-3.1-8B as a query generator to create preference pairs.
3. **w/SFT-T5:** Using a supervised fine-tuned google/flan-t5-small, trained on question-query pairs, as the query generator to create preference pairs. (see Appendix A)
4. **w/SELF-GEN:** Using a self-generating reinforcement learning loop, where preference pairs are dynamically generated per HotpotQA question during training.

4.4 Evaluation Metrics

We evaluate our experiments using the following metrics:

- **Average Precision (AP):** Measures the precision at each rank position where a relevant document is retrieved and averages these values, rewarding methods that rank relevant documents higher in the list.
- **Precision:** Calculates the fraction of retrieved documents that are relevant, without considering their order in the ranked list.
- **Recall@k:** Measures the fraction of gold supporting documents that appear within the top- k retrieved results.
- **F1 Score:** The harmonic mean of precision and recall for retrieved supporting documents compared to the gold supporting documents.

Method	AP	F1	Prec.	Rec.
FEW-SHOT ONLY	0.55	0.30	0.22	0.47
w/T5	0.56	0.29	0.21	0.46
w/LLAMA3-8B	0.54	0.29	0.21	0.47
w/SFT-T5	0.62	0.34	0.25	0.55
w/SELF-GEN	0.55	0.29	0.21	0.46

Table 1: Retrieval performance of hop 1

Method	AP	F1	Prec.	Rec.
FEW-SHOT ONLY	0.48	0.28	0.20	0.45
w/T5	0.48	0.28	0.20	0.45
w/LLAMA3-8B	0.46	0.28	0.20	0.45
w/SFT-T5	0.56	0.32	0.23	0.51
w/SELF-GEN	0.48	0.29	0.21	0.47

Table 2: Retrieval performance of hop 2

5 Results and Discussion

Method	AP	F1
FEW-SHOT ONLY	0.51	0.29
w/T5	0.52	0.29
w/LLAMA3-8B	0.50	0.29
w/SFT-T5	0.59	0.33
w/SELF-GEN	0.52	0.29

Table 3: Average retrieval performance across hops

Our experiments show that the preference dataset generated using the fine-tuned T5 model (SFT-T5) achieves the best performance, improving the Average Precision (AP) score by about 8% compared to other methods. We believe this improvement comes from the fine-tuning process: training the query generator on 100 question-query pairs helps it learn more effective query patterns. As a result, it produces higher-quality preference pairs, which lead to better performance during IPO training.

In contrast, using LLaMA3-8B does not lead to performance gains, despite its larger size. We attribute

this to its general-purpose nature since it is not fine-tuned for multi-hop query generation, its queries are less focused on HotpotQA tasks. Similarly, the self-generating RL loop shows minor improvements. We suspect this is because starting without a well-trained model leads to inconsistent preference pairs, which negatively affects learning.

6 Future Work

Future work can focus on improving both prompting strategies and the optimization process. One idea is to use DSPy’s BootstrapFewShotWithRandomSearch to create more diverse and effective prompts to use in preference dataset creation. Another direction is to build preference datasets in bigger batches during the self-generation loop and update the model iteratively, which could make training more stable and provide better insights. We also plan to explore assigning different weights to each hop in the IPO loss to better balance the importance of different steps in multi-hop retrieval.

7 Conclusion

In this work, we introduced a trajectory-aware reinforcement learning framework for multi-hop query generation, focusing on lightweight models such as Flan-T5-small. By using preference-based optimization through Identity Policy Optimization (IPO), we demonstrated that our approach can improve retrieval performance by up to 8 percentage points in Average Precision compared to few-shot baselines. Our results indicate that a carefully fine-tuned small model can outperform larger general-purpose models like LLaMA3-8B for this task, underscoring the benefits of task-specific adaptation. Given its demonstrated potential for improvement, a small model like T5 could achieve performance comparable to larger models while being significantly more efficient. We believe that the insights from this study can guide future research on developing cost-effective and efficient retrieval-augmented generation pipelines.

8 Implementation Details

We ran all experiments using Google Colab environments with A100 GPUs. The A100, with 40 GB of VRAM, is required to use meta-llama/Llama-3.1-8B. The development environment is based on Python 3.11. We primarily use the HuggingFace *transformers* library for models and generation, and the HuggingFace *datasets* library for loading and processing the HotpotQA dataset. For optimization and model training, we use PyTorch as the underlying deep learning framework.

For IPO training, we used the AdamW optimizer with a learning rate of 5×10^{-6} , a batch size of 1, and trained for 1 epoch. We set $k = 5$ for both candidate query generation and Recall@k evaluation. Unless otherwise specified, we use $\tau = 0.05$ and set the discount factors to $\gamma_1 = 1.0$ and $\gamma_2 = 0.7$.

Appendix

A SFT of Flan-T5

We fine-tuned Flan-T5-small on 100 question-query pairs sampled from HotpotQA. During training, we used a 3-shot prompting strategy: for each training instance, three question-query pairs were randomly selected as examples, and the model was trained to generate the query for the fourth question, with the prompt in Figure 2. This training was done for 20 epochs to allow the model to learn structural patterns specific to multi-hop query generation.

This fine-tuned model (SFT-T5) was then used to generate higher-quality queries for building the preference dataset, which showed significant improvements in retrieval performance.

B Prompt Template

```
Question: {ex.question.1}
Query: {ex.query.1}

Question: {ex.question.2}
Query: {ex.query.2}

Question: {ex.question.3}
Query: {ex.query.3}

Context: {context} // if exists

Question: {question}
Query:
```

Figure 2: Prompt template used for query generation

References

- [1] Sheryl Hsu, Omar Khattab, Chelsea Finn, and Archit Sharma. Grounding by trying: LLMs with reinforcement learning-enhanced retrieval, 2024.
- [2] Jerry Huang, Siddarth Madala, Risham Sidhu, Cheng Niu, Julia Hockenmaier, and Tong Zhang. Rag-rl: Advancing retrieval-augmented generation via rl and curriculum learning, 2025.
- [3] Zheng Wang, Shu Xian Teo, Jieer Ouyang, Yongjun Xu, and Wei Shi. M-rag: Reinforcing large language model performance through retrieval-augmented generation with multiple partitions, 2024.
- [4] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov,

nov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018.

- [5] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences, 2023.
- [6] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.
- [7] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. Colbertv2: Effective and efficient retrieval via lightweight late interaction, 2022.
- [8] Yiqun Chen, Lingyong Yan, Weiwei Sun, Xinyu Ma, Yi Zhang, Shuaiqiang Wang, Dawei Yin, Yiming Yang, and Jiaxin Mao. Improving retrieval-augmented generation through multi-agent reinforcement learning, 2025.
- [9] Yuelu Ji, Zhuochun Li, Rui Meng, Sonish Sivarajkumar, Yanshan Wang, Zeshui Yu, Hui Ji, Yushui Han, Hanyu Zeng, and Daqing He. Rag-rlrc-laysum at biolaysumm: Integrating retrieval-augmented generation and readability control for layman summarization of biomedical texts, 2024.